

Limit Order Book & Matching Engine

Building an electronic exchange from first principles — and understanding every part of why

This document exists so that you can explain this project to somebody else. It assumes you know how to program and nothing whatsoever about finance. Every term is defined the first time it appears, every formula is taken apart piece by piece, and nothing is asserted without a reason.

Internal codename · PokéExchange

26 August 2026

Read this before we write any code

PART 0	What problem does an exchange actually solve?
---------------	---

PART 1	Orders, the book, and the vocabulary
---------------	--------------------------------------

PART 2	Matching — the fifteen lines at the centre
---------------	--

PART 3	The fiddly rules, and why each one exists
---------------	---

PART 4	Why this is a hard engineering problem
---------------	--

PART 5	The mathematics, explained slowly
---------------	-----------------------------------

PART 6	Making the Pokémon layer do real work
---------------	---------------------------------------

PART 7	What we build, in what order, and why
---------------	---------------------------------------

GLOSSARY	Every term, one line each
-----------------	---------------------------

What problem does an exchange actually solve?

Before any code, the human problem. Everything the engine does is a mechanical answer to a difficulty that existed long before computers.

Suppose you own a Charizard and you want money instead. You have a problem with three separate parts, and they are worth naming because the exchange solves each one with a different piece of machinery.

First, search. Somewhere out there is a person who wants a Charizard more than they want their money. You do not know who they are. Finding them costs you time, and the longer it takes, the more likely you accept a worse deal out of sheer impatience.

Second, price. Even once you find them, neither of you knows what a Charizard is *worth*. You will haggle. The outcome will depend on who is more desperate and who is better at haggling — not on any underlying truth. Two identical cards will sell for different amounts on the same afternoon.

Third, trust. You must hand over the card before, after, or at the same moment as they hand over the money. Somebody has to go first, and going first is dangerous.

The exchange's answer

An exchange is a single public place where everyone who wants to buy or sell posts their intentions in a standard format. That one move dissolves all three problems at once. Search collapses, because everyone is already in the room. Price becomes a public fact rather than a private negotiation, because you can see what everyone else is willing to pay. And trust stops mattering, because you are not dealing with a stranger — you are dealing with the exchange, which guarantees the swap.

THE SINGLE MOST IMPORTANT IDEA IN THIS DOCUMENT

The exchange never trades. It owns nothing, wants nothing, and has no opinion about whether Charizard is expensive. It is a referee. Its only job is to take everybody's stated intentions and pair them up according to rules that were published in advance, so that nobody can afterwards claim they were cheated.

Every design decision in the rest of this document is a consequence of three words: **fair**, **fast**, and **auditable**. When you cannot remember why we did something, it will be one of those three.

Two ways to run the room

You could collect everybody's intentions all morning, then at noon work out the single price that lets the most trades happen, and execute everything at once. That is a **call auction**, and real exchanges genuinely do this at the open and close of each day. It is fair and it is efficient, but you can only trade at noon.

Or you could match people the instant a compatible pair exists, continuously, all day. That is a **continuous double auction** — “double” because both buyers and sellers post prices, rather than one side setting prices and the other taking them. This is what essentially every modern market does, and it is what we are building. The cost of continuous trading is that *time now matters*: if two people want the same thing, the one who asked first should get it. Hold onto that, because it is about to become the central rule of the whole system.

Orders, the book, and the vocabulary

Six words — bid, ask, spread, mid, depth, liquidity — carry most of the meaning in this field. Here they are, with pictures.

An order is a sentence with four parts

Everything a participant can say to the exchange reduces to one sentence: *"I want to **buy or sell**, this **many**, at this **price** or better, under these **conditions**."* That is it. There is no richer language. The two important kinds:

A limit order names a price and waits. "Buy 100 at 99" means *buy 100 for me, but never pay more than 99*. If nobody will sell that cheaply right now, the order does not fail and it does not compromise — it **rests** in the book and waits, possibly forever. A limit order is patience written down.

A market order names no price. "Buy 100, now" means *get me 100, I accept whatever it costs*. It never rests, because there is nothing to wait for; it consumes whatever is available and stops. A market order is urgency written down.

WHY THE ASYMMETRY MATTERS

A limit order can be left unfilled but never gives a bad price. A market order is always filled but can give a terrible price. You are choosing which risk you would rather carry — **price risk or execution risk** — and you cannot avoid both. Essentially all of trading strategy is that one trade-off, applied over and over.

The book is just the pile of unmatched patience

At any instant, some limit orders have not found a counterparty. Sort them by price and you have the **limit order book**: buyers on one side, sellers on the other, and a gap in the middle where nobody agrees.

Figure 1 — A limit order book



Every bar is a queue of real people waiting to trade at that price. Buyers stack up below, sellers above, and the untraded gap between the two best prices is the spread. Depth grows as you move away from the middle: a few brave orders near the spread, a wall of patient ones behind.

Read Figure 1 once more, slowly, because five of our six words are visible in it:

WORD	MEANS	IN FIGURE 1
Bid	a price someone will buy at	the blue side. Buyers naturally want to pay <i>less</i> , so bids sit below.
Ask (or offer)	a price someone will sell at	the red side. Sellers want to receive <i>more</i> , so asks sit above.
Best bid / best ask	the highest bid, the lowest ask	99 and 101 — the two prices closest to the gap. Together they are called the <i>top of book</i> , and they are the only two prices most people ever look at.
Spread	best ask – best bid	101 – 99 = 2 ticks . The width of the disagreement.
Mid	the average of the two	100. Not a price anybody is offering — a convention, our best single guess at “the” price.
Depth	how much size is resting	the bar lengths. 50 at the best ask, but 400 three ticks up.

Why the spread cannot be zero

A natural question: if buyers want to buy and sellers want to sell, why is there a gap at all? Because the best bid being *below* the best ask is not an accident — it is a mathematical necessity. If the best bid ever rose to meet the best ask, those two orders would immediately trade and vanish, reestablishing the gap. A book with a bid at or above the ask is called **crossed**, and it is a state that can only exist for the instant before the engine resolves it.

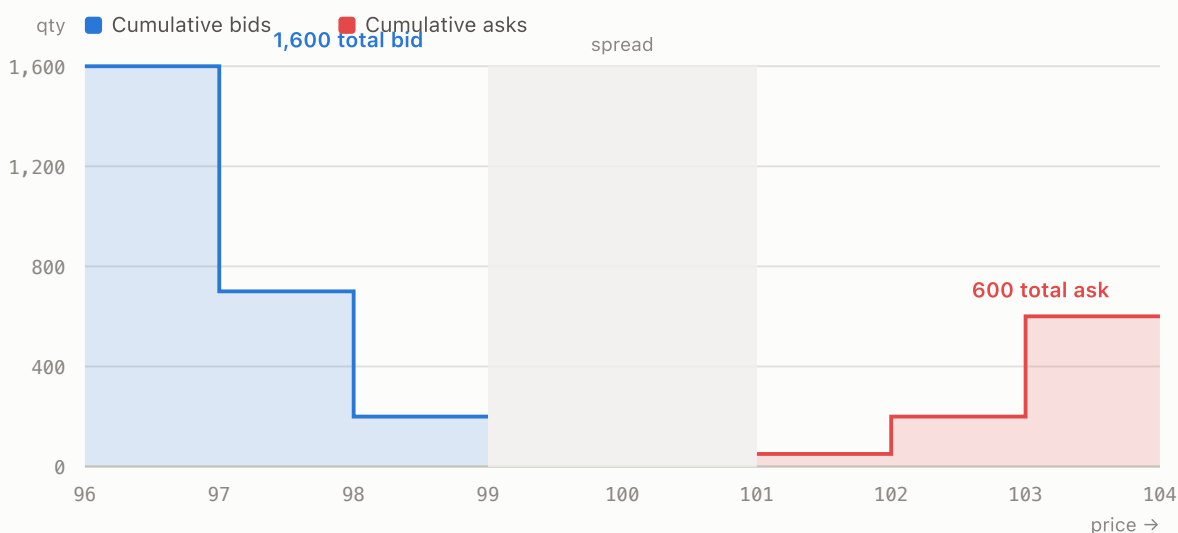
OUR FIRST HARD INVARIANT

best bid < best ask, always, at every moment an outside observer can look. This is one of the properties our fuzz tests will assert after every single message. If we ever violate it, the engine is broken — not slow, not suboptimal, *broken*, because it means we failed to execute a trade that both parties had already agreed to.

The same book, seen as a cost curve

The ladder tells you what is available at each price. Often the more useful question is cumulative: *if I keep buying until I have spent my way up to price P, how much will I have got?* That is the depth curve.

Figure 2 — The same book as a depth curve



Each step answers one question: "if I bought/sold everything up to this price, how much would I get?" The steeper the wall, the harder the book is to push. A thin, flat curve — a Legendary with almost no float — means even a small order rips through several price levels.

Liquidity, defined properly

Liquidity is the most abused word in finance. Used precisely, it means: *the ability to trade a meaningful size, quickly, without moving the price much*. It has three independent dimensions, and Figure 2 shows two of them directly:

- **Tightness** — how narrow the spread is. The cost of a tiny trade.
- **Depth** — how much size sits at each level. The cost of a large trade.
- **Resiliency** — how fast the book refills after being eaten. Not visible in a snapshot; you need to watch the book over time. This one only becomes measurable once we have agents, in sub-project B.

A market can be tight but shallow — a one-tick spread with only 5 units behind it, which is wonderful if you want 5 and useless if you want 500. This is precisely the situation we will engineer deliberately with Legendaries: a tiny float, a razor-thin book, and a chance to watch what happens to anyone who tries to trade size in it.

THE BARGAIN AT THE HEART OF EVERY MARKET

Someone posting a limit order is **providing** liquidity: they are standing there offering everyone else the option to trade instantly. Someone sending a market order is **taking** it. The provider gets paid for that service — in expectation, roughly half the spread — and in exchange accepts the risk of being on the wrong side when the price moves. The taker pays the spread and buys certainty.

The spread is therefore not a fee, and it is not friction. **It is the price of immediacy**, set by competition between the people willing to sell it.

Matching — the fifteen lines at the centre

The part everyone assumes is complicated. It is not. What is hard is making it fast, and that is Part 4's problem.

The fairness rule: price, then time

When an incoming order could trade against several resting orders, which one wins? The rule is **price-time priority**, and it has exactly two clauses applied in order:

1. **Better price first.** If you are selling, whoever bids most gets served first. This is not charity, it is efficiency: it guarantees trades happen at the best available price, which is the entire point of gathering everyone in one room.
2. **Then earlier arrival.** Among orders at *the same* price, the one that has been waiting longest goes first. Strict first-in-first-out.

Clause two is the interesting one, because it is a pure fairness rule with no efficiency justification — the exchange gets an identical outcome either way. It exists because a queue that anyone can jump is a queue nobody will join, and an exchange with no resting orders is not an exchange. **Time priority is the payment for patience**, and it is what makes providing liquidity worth doing at all.

Watching an order eat the book

Now the mechanism. An incoming order that can trade immediately is called **aggressive** or a **taker**; the resting orders it trades against are **passive**, or **makers**. The taker walks the opposite side of the book, cheapest first, printing a trade at each stop, until it is satisfied or the book runs dry.

Figure 3 — Matching, one fill at a time

A market order to BUY 180 walks up the ask side, oldest order at each price first

① Book before

103 400
102 150
101 50

PRINTS ON THE TAPE

—

② Fills 50 @ 101

103 400
→ 102 150
101 gone

PRINTS ON THE TAPE

50 @ 101

③ Fills 130 @ 102

103 400
→ 102 20
101 gone

PRINTS ON THE TAPE

50 @ 101
130 @ 102

Result: 180 bought. Average price paid = $(50 \times 101 + 130 \times 102) / 180 = 101.72$

The buyer wanted 101 and got 101.72. That 0.72 is slippage — the cost of being in a hurry.

This is the entire engine. The incoming order (the taker) is matched against resting orders (the makers) in strict price-then-time order, printing a trade for each pairing, until it is filled or the book runs out.

The single most-failed detail: whose price wins?

In Figure 3 the buyer bought at 101 and then at 102. Suppose instead of a market order they had sent a *limit* order to buy 180 at 105. What price should the first 50 trade at — 101, or 105?

101. Always the resting order's price, never the incoming order's. The reasoning is clean: the resting seller published terms ("I will sell at 101") and has been standing behind them, in public, waiting. The arriving buyer read those terms and chose to accept them. You do not get to charge somebody more than they advertised because you discovered they would have paid it.

The buyer therefore receives four ticks of **price improvement** — they were willing to pay 105 and paid 101. This is a real and routine occurrence, it is the direct reward for posting a wide limit rather than a market order, and getting it backwards is the most common bug in a first matching engine.

Partial fills and residuals

In panel ③ the incoming buyer needed 130 but the level held 150. The resting order is filled for 130 and **remains in the book with 20 left**, keeping its original place in the queue — it did nothing wrong, so it loses nothing. Correspondingly an order can be filled in twenty pieces at five different prices over an hour. So every order carries two quantities forever: what it originally asked for, and how much is still outstanding. Confusing those two is bug number two.

The whole algorithm

Here it is in full. Not a sketch — this is genuinely the complete description of what a matching engine does.

```
// an order arrives to BUY `qty` at `limit`
while remaining > 0
  and book has an ask
  and best_ask <= limit:                // is there a price we can accept?

    resting = front of level[best_ask]    // front == oldest == time priority
    fill    = min(remaining, resting.remaining)

    print TRADE at best_ask for fill      // the MAKER's price
    remaining      -= fill
    resting.remaining -= fill

    if resting.remaining == 0: remove it from the queue
    if level is now empty:    drop the level

if remaining > 0:                        // we could not fill it all
  LIMIT -> insert at back of level[limit] // join the queue and wait
  MARKET -> cancel the remainder         // nothing left to hit
```

WHERE THE DIFFICULTY ACTUALLY LIVES

Read that loop and count the operations it needs: **find the best price, get the front of a queue, remove from a queue, insert at the back of a queue**. Four operations. Plus one more that does not even appear above: **find an arbitrary order by its ID**, which is what cancelling needs.

Those five operations are the entire engineering project. The loop above will be written once, in an afternoon, and never meaningfully change again. Parts v0 through v4 of our build are five different attempts to make those five operations faster, and nothing else.

One more requirement: it must be repeatable

An exchange settles real obligations, so “why did my order not fill?” must have an exact answer, reproducible months later. That gives us the strongest constraint in the whole design: **determinism**. The same sequence of input messages must produce a byte-for-byte identical sequence of output events, on any machine, forever.

Which forbids more than you would expect. No reading the wall clock inside the engine — time priority is established by an integer counter that ticks once per accepted message, not by a timestamp. No random numbers. No iterating a hash map whose order depends on where the allocator happened to put things.

No relying on which thread got there first. Determinism is not a testing convenience we are being fastidious about; it is the property that makes the two most valuable things in the project possible at all:

- **Differential testing.** Write the engine twice — once carefully in C++, once simply in Python — feed both the same million random messages, and demand identical output. Two independent implementations agreeing is a far stronger correctness claim than any hand-written test suite.
- **An honest benchmark.** To claim v3 is faster than v0 we must prove they did the *same work*. Determinism lets us assert they produced identical trades, so the only thing that differed was the speed.

The fiddly rules, and why each one exists

Every rule below was invented in response to somebody getting hurt. Knowing the injury is how you remember the rule.

The order types

TYPE	BEHAVIOUR	THE PROBLEM IT SOLVES
Limit	Trade what you can at your price or better; rest the remainder.	The default. Never a bad price, but possibly no fill.
Market	Trade until filled, at any price; cancel any remainder.	Certainty of getting done. Dangerous in a thin book — see Part 5 on slippage.
IOC immediate-or-cancel	Trade what you can right now at your price; cancel the rest instantly rather than resting.	“Take what’s there, but do not leave me exposed.” A resting order broadcasts your intentions to everyone. IOC lets you take liquidity without ever showing your hand.
FOK fill-or-kill	Fill the <i>entire</i> quantity immediately or do nothing at all. All or nothing.	When a partial fill is worse than no fill — e.g. one leg of a multi-instrument trade. Buying half a hedge leaves you more exposed than buying none of it.
Post-only	Must rest. If it would trade immediately, reject it instead.	Guarantees you are the maker, never the taker. Matters because exchanges usually charge takers and pay makers a small <i>rebate</i> ; a post-only order is a promise you will not accidentally pay the fee.

Cancel and modify: the interview question

Cancel is simple to state (“remove order 8,271,993”) and, as Part 4 will show, the single most performance-critical operation in the entire system. Modify is where the interesting question lives. Say you have an order resting in the queue and you want to change it. Do you keep your place in line?

YOU WANT TO...	PRIORITY	BECAUSE
Reduce the quantity	Keep it — stay where you are	You are asking for <i>less</i> . Nobody behind you is harmed; in fact everyone behind you moves closer to the front. There is no one to compensate, so there is nothing to charge you.
Increase the quantity	Lose it — go to the back	The extra quantity never waited. If you kept your slot you would be inserting brand-new size ahead of people who have been queuing — a strictly better deal than they got, for free. That is queue-jumping.
Change the price	Lose it — back of the new level	Priority is a property <i>of a price level</i> , not of you. You have never queued at the new price for even a microsecond.

HOW TO ANSWER THIS IN AN INTERVIEW

Do not recite the table. State the principle and derive the table from it in real time: **time priority is compensation for exposure risk, so you keep priority exactly as long as you have not increased the risk anyone else is taking on your behalf**. Reducing quantity lowers your exposure; everything else raises it.

Then add the implementation note, because it is what they are really probing: “which means a reduce is an in-place decrement of one field, but an increase or a reprice is a cancel followed by a fresh insert — so I implement modify in terms of the two primitives I already have, rather than as a third code path that can disagree with them.”

Self-trade prevention

Nothing stops your own buy order from matching your own sell order. The trade is economically meaningless — you pay yourself — but it is not harmless. It prints on the public tape, so it manufactures the appearance of activity that never happened. Done deliberately and repeatedly, that is **wash trading**: illegal market manipulation, because it fakes the volume signal other participants rely on. Exchanges therefore refuse the match outright. Which of the two orders gets cancelled is a published policy choice; we will cancel the incoming one, as it is the simpler rule to reason about and to test.

Rejections are data, not exceptions

Bad input — an unknown instrument, a zero quantity, a cancel for an order that already filled — produces a **Rejected** event carrying a reason code, and the book is left completely untouched. The engine never throws and never logs.

Two reasons, and they are worth understanding because this is the kind of decision that reads as pedantic until it saves you. First, a function that can throw halfway through a match can leave the book half-updated, and a half-updated book is not recoverable — you have lost the audit trail, which was the whole point. Second, an error path that is *data* gets exercised by the same tests as everything else, whereas an error path that is an exception is the code nobody ever tests. Malformed input is not an exceptional circumstance in a system open to the public. It is Tuesday.

Why this is a hard engineering problem

The algorithm is fifteen lines. The performance work is weeks. This part explains that gap, because the gap is the reason the project is worth building.

The traffic is not what you would guess

Ask someone to guess the message mix at a real exchange and they will say mostly orders, some trades, a few cancels. It is almost exactly inverted. A representative mix looks like this:

MESSAGE	SHARE	CONSEQUENCE FOR US
Cancel	~90%	Automated strategies re-quote constantly — post, reconsider, pull, repost, thousands of times a second. This is the operation to optimise.
New order	~9%	Needs a fast insert at the back of a queue.
Actual trade	~1%	The rarest event in the system, and the one everyone thinks about first.

Sit with that for a second, because it inverts the natural design instinct. If you set out to build a matching engine you would optimise *matching*. But roughly nine in ten messages never touch the matching loop at all — they are somebody saying “forget order 8,271,993”. So the hot path is not the loop from Part 2. The hot path is **find one order among millions, by ID, and unlink it — right now**. Everything below follows from that sentence.

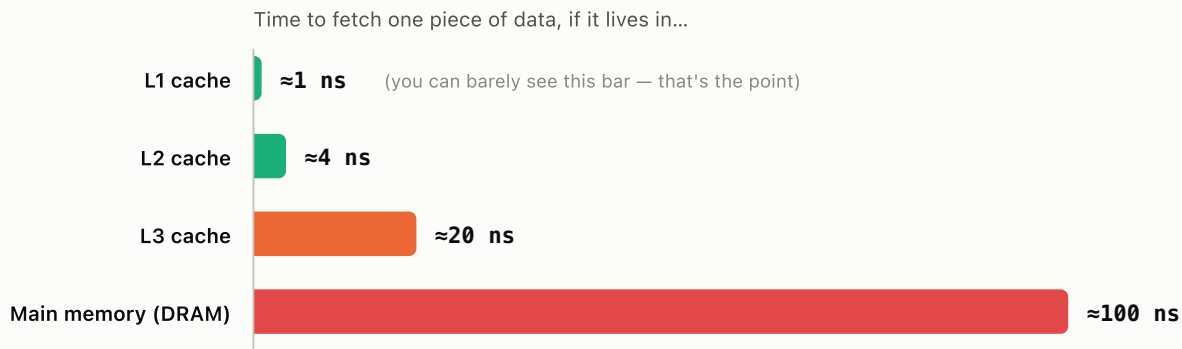
CAVEAT ON THESE FIGURES

The 90/9/1 split is the widely-reported shape of message traffic on major electronic venues, quoted here as an order of magnitude rather than a precise statistic — exact ratios vary enormously by venue, instrument and year. We should cite a specific published figure in the final README rather than this approximation, and we will be able to measure our *own* mix exactly, which is what the benchmark actually needs.

The reason fast code is not about instruction count

Here is the fact that reorganises how you think about performance. A modern CPU core executes several billion instructions per second. It can also be made to sit completely idle for a hundred nanoseconds — several hundred instructions’ worth of time — waiting for a single number to arrive from memory.

Figure 4 — Why memory layout is the whole game



A cache miss costs ~100× a cache hit. This single fact is why v1–v3 exist.

Modern CPUs are starved, not slow. A processor can do a hundred arithmetic operations in the time it takes to wait for one value that isn't in cache. Order-of-magnitude figures for a typical desktop CPU.

The CPU never fetches one value. It fetches a 64-byte **cache line** and hopes you wanted the neighbours too. So the question that determines your speed is not *how many operations do I perform*, it is **how often is the thing I need next already sitting next to the thing I just touched**. Which makes it a question about memory *layout*, not about algorithms.

This is why `std::map` — a perfectly good, $O(\log n)$, textbook-correct data structure — is the wrong tool. Each node was allocated separately, whenever it happened to be created, so the nodes are scattered across memory. Walking from one price level to the next is a **pointer chase**: read a pointer, wait for memory, read the next pointer, wait again. The asymptotic complexity is fine. The constant factor is a disaster.

Figure 5 — The same book, laid out two ways

v0: std::map<Price, deque<Order>>

each node landed wherever the allocator had room — walking the book jumps all over RAM



v1: Order levels[MAX_TICKS]

price IS the array index — neighbouring prices are neighbouring bytes, and the prefetcher sees you coming



Same algorithm. Same big-O. Roughly 6× fewer trips to memory.

Dashed boxes are 64-byte cache lines: the CPU always fetches a whole line, never one value.

A tree stores its nodes wherever malloc found space, so traversing it is a pointer chase: fetch, wait, fetch, wait. An array indexed by price is contiguous, so the hardware prefetcher loads the next levels before you ask.

And the fix is available to us because of a fact about prices specifically: **prices are discrete**. They move in ticks, and a tick count is an integer. So “the book” can simply be an array with one slot per possible price, and finding a price level becomes address arithmetic — no search, no pointers, no tree. This is the observation the entire optimisation programme rests on, and it is also the reason Part 1’s insistence that prices are integers and never floats was not fussiness. An array cannot be indexed by 100.35.

The five versions, and what each one buys

VER	CHANGE	THE SPECIFIC PROBLEM IT ATTACKS
v0	<code>std::map<Price, deque<Order>></code>	None — it is the honest baseline, and we keep it forever. Without a slow version there is no story, only an assertion.
v1	Array indexed by tick	Pointer chasing and tree rebalancing. Finding a level becomes arithmetic. Best bid/ask tracked with a moving pointer or a bitmask so we never scan for it.
v2	Intrusive linked list + object pool	<code>malloc</code> on the hot path. Every allocation is a function call that may take a lock and may wander off to the OS. We pre-allocate all the order slots once and hand them out from a free list, so steady-state operation allocates nothing at all. "Intrusive" means the list pointers live inside the order object, so an order and its links arrive in the same cache line.
v3	Open-addressing hash: ID → node*	The big one. Cancel, our ~90% case, goes from "search the book" to one hash lookup plus two pointer writes to unlink. Open addressing rather than <code>std::unordered_map</code> because the latter is a linked list per bucket — more pointer chasing, exactly what we are trying to escape.
v4	Lock-free SPSC ring buffer	Stretch goal. Separates the thread reading the network from the thread doing the matching, so neither waits on the other. Requires cache-line-aligning the two shared indices, or the two threads fight over one cache line and you get false sharing — a slowdown with no logical cause, and a genuinely good thing to have debugged once.

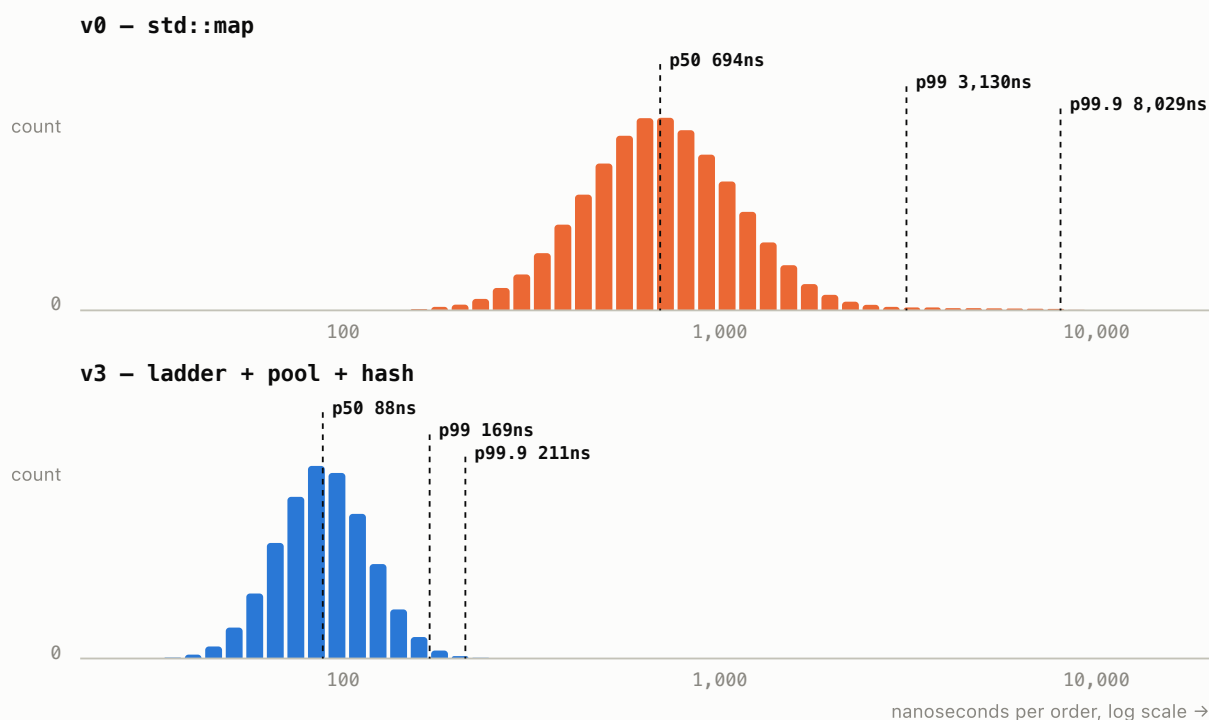
Measuring it without fooling ourselves

A benchmark is a claim, and most benchmark claims are wrong. Ours has to survive someone hostile reading it, so:

- **Report the distribution, never the average.** An average latency hides exactly the thing that matters. If one order in a thousand takes fifty times as long, your mean barely moves — and in a real market that one order is where you get picked off. So we quote **p50, p99, p99.9 and max**, and we show the histogram.
- **Identical input to every version.** One recorded message file, replayed into each version, with the outputs asserted byte-identical. Determinism is what makes this possible.
- **Measure outside the engine.** Timing instrumentation inside the matching loop would change what it is measuring.

- **Discard the warm-up.** The first few thousand messages measure a cold cache and an un-JITed branch predictor, not steady-state behaviour.

Figure 6 — What a latency measurement looks like



Two panels, one shared axis — never two y-scales on one chart. The whole distribution matters, not the average: the long right tail is where money is lost, which is why the numbers you quote are p99 and p99.9.

ILLUSTRATIVE SHAPE ONLY — simulated to show the form of the result. We have not built the engine yet; producing these numbers honestly is the entire point of sub-project C.

A REAL CONSTRAINT WE HAVE TO PLAN AROUND

You are on an Apple-silicon Mac, and two of the tools the spec assumes do not exist there. `rdtsc` is an x86 instruction; the ARM equivalent is reading the `cntvct_el0` system register, which ticks at a different and lower frequency. And `perf stat` — which is how you would get cache-miss and branch-misprediction counts — is Linux-only. macOS offers `xctrace`, which is considerably less direct.

This does not block sub-project A at all, and I have kept the timing abstraction outside the engine so it cannot contaminate the design. But before sub-project C we need to choose: a cheap Linux cloud box for the benchmark runs, or a local VM, or accept weaker profiling evidence on macOS. My recommendation is the Linux box — `perf stat` output showing the cache-miss rate falling between v0 and v1 is one of the most persuasive artefacts this project can produce, and it is worth a few dollars.

The mathematics, explained slowly

Ten pieces of maths, in dependency order. Each one is stated, then taken apart term by term, then justified. None of it requires anything beyond first-year calculus and probability.

5.1 Returns — and why we take logarithms

Prices themselves are nearly useless for analysis: knowing Charizard is at 4,300 tells you nothing without knowing where it was. What carries information is **change**, expressed as a fraction so that instruments of different price can be compared. That is a **return**, and we almost always use the log form:

$$r_t = \ln \frac{P_t}{P_{t-1}}$$

P_t the price now · P_{t-1} the price one step ago · $\ln$ the natural logarithm

Three reasons for the logarithm, and the first is the one that matters:

- **Log returns add up; simple returns do not.** Go up 10% then down 10% and you are at 99, not 100 — so simple returns cannot be summed over time. Log returns can: the return over a week is exactly the sum of the daily log returns. This makes every subsequent formula cleaner.
- **They are symmetric.** A halving and a doubling are -0.693 and $+0.693$. With simple returns they are -50% and $+100\%$, which makes an unbiased process look biased.
- **They approximate percentages for small moves.** $\ln(1.01) = 0.00995$, so for the small moves that dominate tick data you can read a log return as a percentage and be right to three decimal places.

5.2 Volatility — the size of the wiggle

Volatility is simply the standard deviation of returns. It measures how far the price typically moves per unit of time, ignoring direction entirely.

$$\sigma^2 = \frac{1}{n-1} \sum_i (r_i - \bar{r})^2$$

This is the **variance**; volatility σ is its square root. $\bar{r}$ the mean return · n the number of observations · the $n-1$ rather than n is Bessel's correction, because we estimated the mean from the same data

Volatility is the single most important number in this project, because it appears in nearly every formula that follows. It sets how wide a market maker should quote (5.8), how large the price impact of a trade is (5.5), and whether a divergence between two instruments is meaningful or noise (5.10). It is also *not constant* — that fact is a whole stylized fact of its own, in 5.6.

5.3 Mid price, and the better version nobody teaches first

The obvious estimate of “the” price is the midpoint:

$$\text{mid} = \frac{P_{\text{bid}} + P_{\text{ask}}}{2}$$

It is fine, and it is also throwing away information. Look again at Figure 1: 200 resting on the bid at 99, but only 50 on the ask at 101. Far more people want to buy than to sell. The mid says 100, but 100 is clearly not where this book is heading. The **microprice** (or weighted mid) captures that:

$$\text{micro} = \frac{P_{\text{bid}} \cdot Q_{\text{ask}} + P_{\text{ask}} \cdot Q_{\text{bid}}}{Q_{\text{bid}} + Q_{\text{ask}}}$$

Note the **crossed** weighting: each price is weighted by the quantity on the *opposite* side.

The crossing looks like a typo and is the entire idea. A large bid quantity is evidence of buying pressure, so it should pull the fair value *up*, towards the ask. Working Figure 1 through: $(99 \times 50 + 101 \times 200) / 250 = 100.6$ — noticeably above the mid of 100, which is exactly right, because that book is lopsided towards buyers.

WHY THIS MATTERS TO US

Any agent that quotes around the plain mid in a lopsided book is systematically quoting too low on one side, and will be picked off there. The microprice is a one-line change that makes our market maker meaningfully less naive, and it is the kind of detail that shows you have thought about the book rather than just read about it.

5.4 What a trade actually costs: VWAP and slippage

Return to Figure 3 — the market order to buy 180. It did not trade at one price; it traded 50 at 101 and 130 at 102. The single number describing what it paid is the **volume-weighted average price**:

$$\text{VWAP} = \frac{\sum_i p_i q_i}{\sum_i q_i} = \frac{50 \times 101 + 130 \times 102}{180} = \frac{18,310}{180} = \mathbf{101.72}$$

Slippage is the gap between the price you hoped for and the VWAP you got. Measured against the mid of 100 it is 1.72 ticks; measured against the best ask of 101 it is 0.72. Both are quoted in practice, so always say which. That 0.72 is the cost of size — the first 50 got the good price and the next 130 did not.

THIS IS THE LEGENDARY EXPERIMENT IN ONE NUMBER

Slippage is determined entirely by the shape of the depth curve. Give a Legendary a float of a few hundred units and its book will be three levels deep, so an order of any consequence sweeps through all of them and the slippage is enormous. Same engine, same order, wildly different cost — because of nothing but market structure. This is the microstructure lesson the theme lets us make visible.

5.5 Market impact: the square-root law

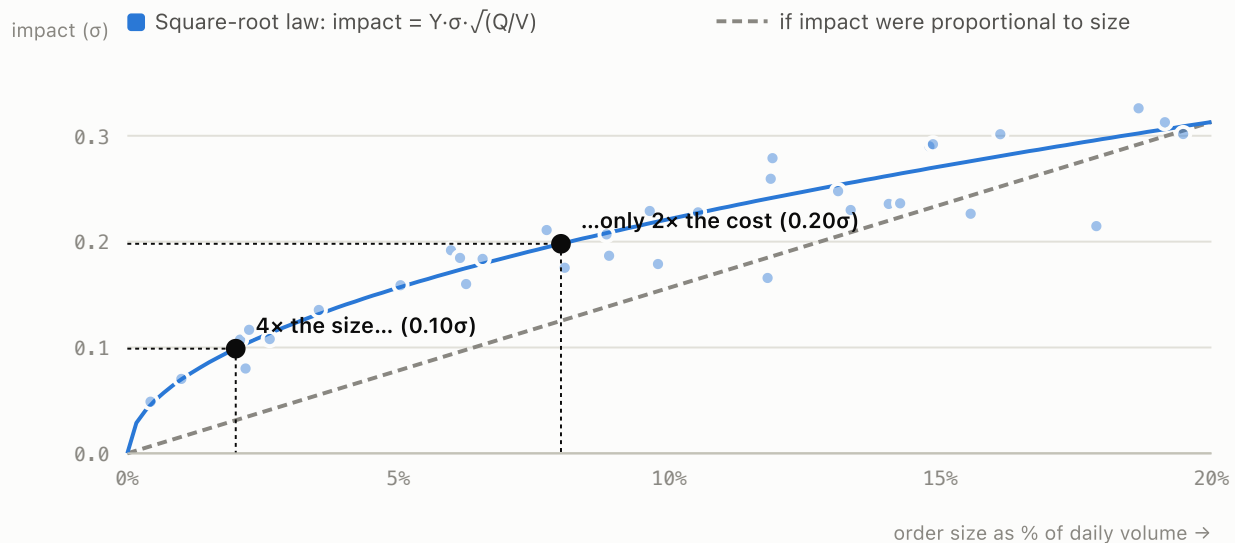
Slippage measures what one order paid. **Impact** asks the broader question: how far does the price *move* when someone trades size Q ? The empirical answer is remarkably consistent across every market anyone has measured, and it is not linear:

$$\Delta P \approx Y \cdot \sigma \cdot \sqrt{Q / V}$$

Q your order size · V the instrument's total volume over the same period · σ its volatility · Y a dimensionless constant, empirically around 0.5–1

Read the formula in words: *impact is volatility, scaled by the square root of the fraction of the market you are*. The square root is the surprising part and the useful part.

Figure 9 — Trading costs grow like a square root, not a straight line



One of the most robust empirical laws in finance, found across every market anyone has checked. It is why large orders are sliced up and fed in slowly, and it is a law our simulated market either reproduces or does not.

Curve is the theoretical law; dots are simulated observations scattered around it, showing the noise a real measurement has.

Why a square root and not a straight line? There is no single agreed derivation, which is itself worth knowing — this is an empirical law that theory has chased rather than predicted. The most intuitive account: the book is not a fixed queue you consume, it is a population of latent traders who react. As you buy, some sellers who were not previously interested are drawn in by the higher price. Your own trading partly refills the book ahead of you, so the marginal cost of each extra unit falls. Concavity follows.

The practical consequence is enormous and is why execution is a profession: splitting one large order into many small ones spread over time genuinely costs less than sending it at once, because you keep re-entering at the cheap end of the curve. Whether our simulated market reproduces a square root — rather than a line, or some other exponent — is one of the sharpest tests of whether the agent population is behaving like real people.

5.6 Stylized facts: the fingerprints of a real market

Financial return series, across every asset class and century anyone has checked, share a set of statistical properties. They are called **stylized facts**. They matter to us for a specific reason: **we are not going to programme them in**. They should *emerge* from agents interacting. If they do, our simulation has captured something true about how markets work. If they do not, we have built an expensive random number generator.

Fact 1 — Returns are fat-tailed

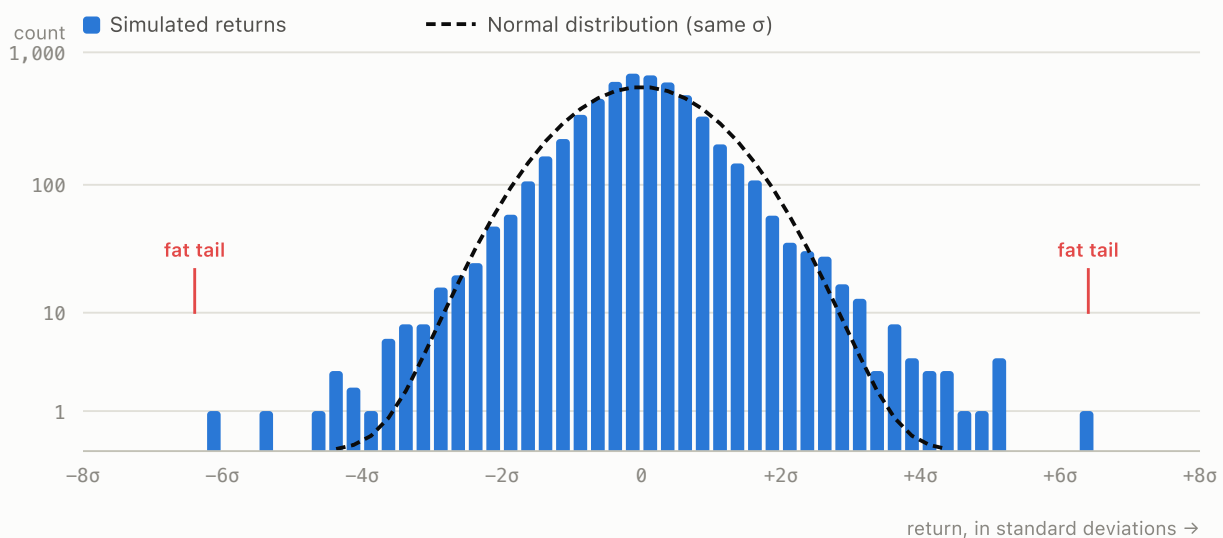
Extreme moves happen far more often than a bell curve allows. The standard measure is **kurtosis**, the normalised fourth moment:

$$\kappa = \frac{E[(r - \mu)^4]}{\sigma^4}$$

A Normal distribution has $\kappa = 3$ exactly. Anything above 3 has fatter tails; financial returns typically land between 4 and 20.

The fourth power is what makes this a tail measure: a move of 5σ contributes 625 times as much as a move of 1σ , so κ is dominated almost entirely by the rare extremes and barely notices the ordinary days.

Figure 7 — Real markets are not bell curves



Log vertical scale. The dashed line is what a Normal distribution predicts; the bars are what actually happens. Beyond about $\pm 4\sigma$ the Normal says "never", and reality says "Tuesday". Measured kurtosis here is 5.81 against 3.00 for a Normal.

Computed from a simulated GARCH(1,1) series with Student- t shocks — the same statistical test we will run against our engine's real output.

The chart above is a genuinely simulated series, and its kurtosis is **5.81** against 3.00 for a Normal. Notice what that means on a log scale: out past $\pm 5\sigma$ the dashed Normal curve has effectively hit zero — it is claiming such days are impossible — while the bars are still clearly there. Assuming normality is not a small error at the tails. It is a categorical one, and it is the mistake behind a good number of famous blow-ups.

Fact 2 — Direction is unpredictable, but size is not

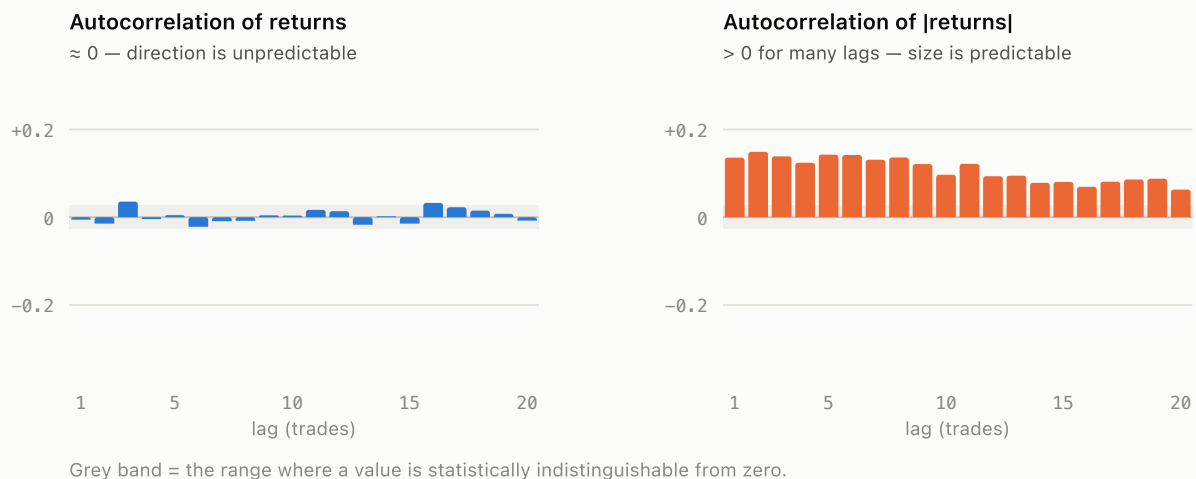
The tool for both halves of that sentence is the **autocorrelation function** — how much a series resembles itself, shifted by k steps:

$$\rho(k) = \frac{\sum_t (r_t - \bar{r})(r_{t+k} - \bar{r})}{\sum_t (r_t - \bar{r})^2}$$

$\rho(k) = 1$ means perfectly self-similar at lag k ; 0 means no linear relationship at all.

Apply it twice, and you get one of the most elegant results in the field.

Figure 8 — Volatility clusters; direction does not



The left panel is why you cannot make money by guessing the next tick. The right panel is why you can still make money selling insurance against volatility: quiet periods follow quiet periods, storms follow storms.

- **$\rho(k)$ of returns ≈ 0** for every k . Whether the price just went up tells you nothing about whether it goes up next. This is not an accident, it is an equilibrium: any reliable pattern would be traded away by exactly the arbitrageurs we are going to build. **Market efficiency is a consequence of competition, not an assumption.**
- **$\rho(k)$ of $|returns| > 0$** , and it decays slowly over many lags. The *magnitude* is highly predictable. Big moves cluster with big moves, quiet with quiet. This is **volatility clustering**, and its intuition is simple: the arrival of news is itself bursty, and it takes the market a while to digest each burst.

So you cannot forecast direction but you *can* forecast turbulence — which is precisely why the market maker in 5.8 has σ in its formula and widens its quotes when σ rises. That agent is monetising Fact 2.

5.7 When do orders arrive? Poisson processes

Our noise traders need arrival times. The standard model for “events happening independently at some average rate” is the **Poisson process**:

$$P(N \text{ events in time } t) = \frac{(\lambda t)^k e^{-\lambda t}}{k!}$$

λ the average rate (orders per second) · k the number of events

The property we actually use is the companion result: if events are Poisson at rate λ , the **gaps between them are exponentially distributed** with mean $1/\lambda$. Which gives a one-line generator — draw U uniform on $(0,1]$ and the wait until the next order is:

$$\Delta t = -\frac{\ln U}{\lambda}$$

Why is this the right model? Because of the **memoryless** property: how long you have already waited tells you nothing about how much longer you will wait. That is exactly true when arrivals are a great many independent participants each deciding on their own, which is what a crowd of noise traders is. It is a poor model for *informed* flow, which is bursty and correlated — and that mismatch is a feature, since it gives our momentum and arbitrage agents a genuinely different statistical signature from the background.

5.8 Avellaneda–Stoikov: how to quote when you can lose money

This is the most substantial piece of mathematics in the project, so here is the problem it solves before the formulas.

You are a market maker. You post a bid and an ask, and you earn the spread each time both get filled. Simple enough — so why not quote a one-tick spread and capture enormous volume? Two reasons, pulling in opposite directions:

- **Quote too wide** and nobody trades with you. You earn a large spread on almost no volume.
- **Quote too tight** and you trade constantly — but your fills will not be balanced. A run of buyers leaves you short a pile of Charizard you never wanted, and now you are exposed to the price moving against you. That exposure is **inventory risk**, and it can dwarf everything you earned in spread.

Avellaneda and Stoikov posed this as a stochastic optimal control problem — maximise the expected utility of your wealth at the end of the session, given that the price diffuses randomly and your fill probability declines as you quote further out — and solved the resulting equation. Two results come out, and both are interpretable.

Result 1: quote around your reservation price, not the market's mid

$$r = s - \underline{q \cdot \gamma \cdot \sigma^2 \cdot (T - t)}$$

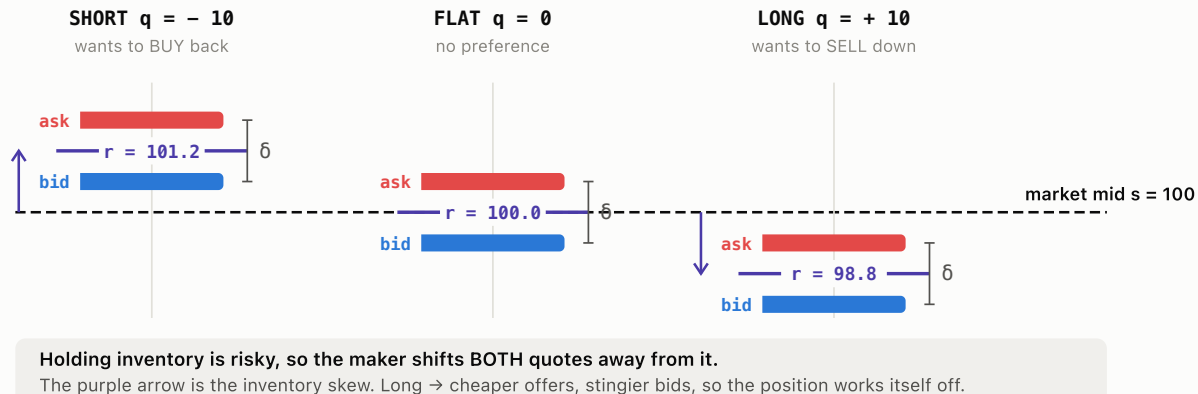
r your reservation price · s the market mid · q your inventory (positive = long) · γ your risk aversion · σ^2 the variance rate · $(T-t)$ time remaining in the session

The underlined term is the whole idea, and every factor in it earns its place:

FACTOR	READ IT AS
q	How much I am already holding. Zero inventory, zero adjustment — and the sign flips when you go short, so the correction always pushes <i>against</i> your position.
γ	How much I mind risk. A γ of zero means you do not care about inventory at all and quote symmetrically around the mid forever.
σ^2	How dangerous holding is <i>right now</i> . In a calm market inventory is nearly free; in a violent one it is terrifying. This is where Fact 2 gets monetised.
$(T-t)$	How long I am stuck with it. Risk is proportional to remaining exposure time, so the adjustment shrinks to zero as the session closes — the model naturally becomes indifferent at the end, because there is no longer time to be hurt.

So: **if you are long, your reservation price is below the market mid.** You are willing to sell a little cheaper than you “should”, and you demand a little more discount to buy more. You have not become bearish; you have priced in the cost of the risk you are already carrying.

Figure 10 — Avellaneda–Stoikov: how inventory bends your quotes



The reservation price r is the price at which the maker is genuinely indifferent — it is not the market mid, it is the mid adjusted for the risk they are already carrying. Quotes sit symmetrically around r , so the skew is what makes the position self-liquidating.

Skew and spread magnitudes chosen for legibility, not calibrated.

Result 2: how wide to make the spread

$$\delta = \underbrace{\gamma \sigma^2 (T - t)} + \underbrace{\frac{2}{\gamma} \ln \left(1 + \frac{\gamma}{k} \right)}$$

then quote $\text{bid} = r - \delta/2$ and $\text{ask} = r + \delta/2$. k measures how sharply your fill rate falls off as you quote further from the mid.

Two terms, doing two different jobs:

- $\gamma \sigma^2 (T - t)$ — the **risk premium**. Identical in form to the inventory skew, and for the same reason: this is what you charge for the danger of holding whatever you are about to be handed. Volatility rises, your spread widens. This is why real spreads blow out during a crash, and it is not panic — it is arithmetic.
- $(2/\gamma) \cdot \ln(1 + \gamma/k)$ — the **market-power term**. It contains no inventory and no time; it depends only on how price-sensitive your counterparties are. It comes from modelling the fill intensity as $\lambda(\delta) = A \cdot e^{-k\delta}$: quote one unit further out and your fill rate drops by a constant factor. A large k means customers are picky and abandon you quickly, so you must quote tight. A small k means they will trade with you almost regardless, so you can charge more. This term is the monopolist's markup.

BE PRECISE ABOUT THIS IN INTERVIEWS

These are the closed-form *asymptotic approximations* from the 2008 paper, not exact solutions — the exact problem is a Hamilton–Jacobi–Bellman PDE, and these expressions come from an expansion of it. Saying “this is the approximate solution to the HJB equation under exponential fill intensity” signals that you read the paper. Saying “this is the optimal spread” signals that you read a blog post about it.

5.9 Adverse selection: why market making is hard

There is a nasty asymmetry hiding in 5.8 that the model handles only indirectly, and it is the single most important concept for understanding why liquidity provision is not free money.

Your bid rests at 99. Who hits it? Disproportionately, **somebody who knows something you do not**. If a Water-type buff is about to be announced, the traders who see it first come and take your offers before you can pull them. Your fills are not a random sample of the market — they are *selected*, and selected against you. Formally, the thing that ruins your day is:

$$E[\Delta s \mid \text{my bid was filled}] < 0$$

In words: conditional on somebody having sold to me, the price is expected to fall. I bought, and then it went down. Every time.

So a market maker’s profit decomposes into two competing pieces:

$$\text{P\&L} = \text{spread captured} - \text{adverse selection} - \text{inventory risk}$$

The first term is small, frequent and reliable. The second is occasional, large and brutal. Market making is the business of collecting the first faster than the second takes it away, and every real market maker’s technology budget goes on reducing the second — which is what all the latency work is ultimately *for*. Being fast is not about greed; it is about being able to withdraw your quotes before the informed trader reaches them.

THE EXPERIMENT THIS MAKES POSSIBLE

This is why the spec has scheduled “meta patches” — announced shocks like *Water types buffed 15%*. If we give agents different reaction latencies, then at each patch the fast agents will systematically profit at the expense of the slow market maker. And because we control everything, **we can measure exactly how much**, in currency, attributable to latency alone. That number is the most quantitatively interesting result the whole project can produce, and it is a direct, honest answer to “why does anybody care about nanoseconds?”

5.10 Cointegration: trading a relationship instead of a price

Charmander, Charmeleon and Charizard are the same creature at three stages. Their prices should be tethered: a Charizard is worth roughly what a Charmander is worth plus the cost of evolving it. Individually each price wanders unpredictably (5.6, Fact 2). But their *difference* should not wander — it should hover around a constant and get pulled back whenever it strays.

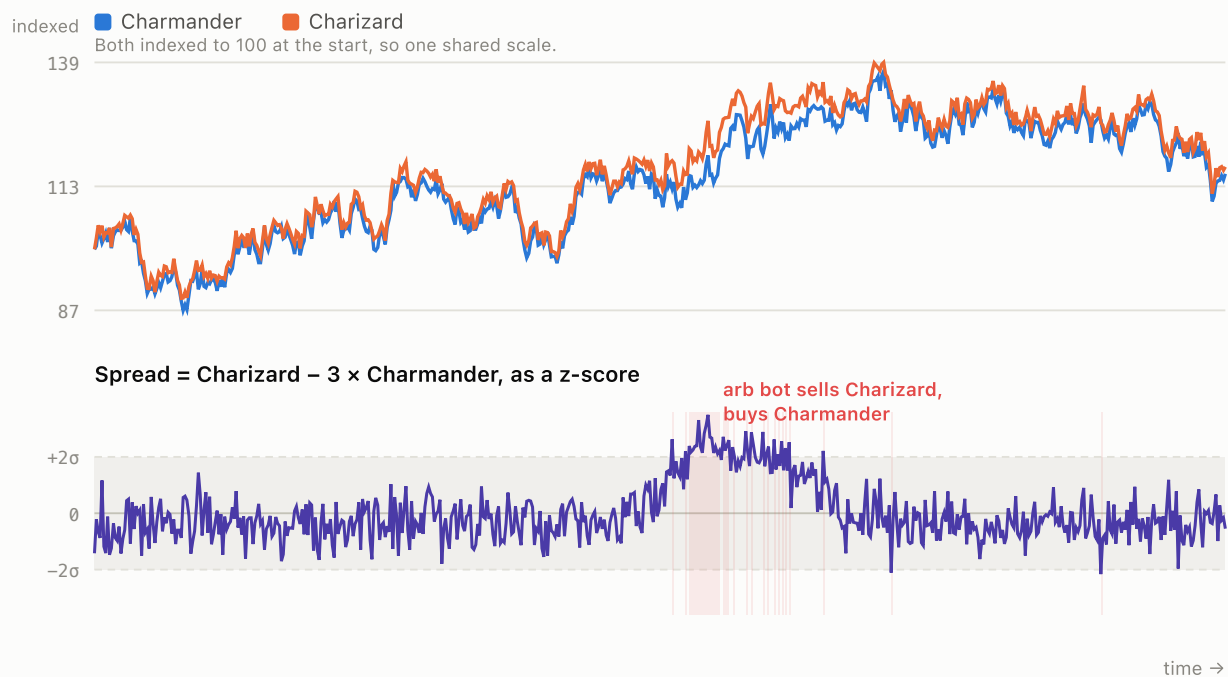
Two series with that property are **cointegrated**. In the standard vocabulary, each price alone is **I(1)** — a random walk, non-stationary, no fixed mean to return to — but some linear combination of them is **I(0)**: stationary, mean-reverting, forecastable.

$$\text{spread}_t = p_t^B - \beta \cdot p_t^A$$
$$z_t = \frac{\text{spread}_t - \mu_{\text{spread}}}{\sigma_{\text{spread}}}$$

β the hedge ratio — how many Charmander one Charizard is worth, estimated by regressing one price on the other · z the spread restated in standard deviations

The strategy is then almost embarrassingly simple: when z climbs above +2, B is expensive relative to A, so sell B and buy β of A. When z falls below −2, do the reverse. Close out when z returns to 0.

Figure 11 — Cointegration: the evolution line has to hang together



Individually each price wanders unpredictably. Their difference does not — it is tethered. When the tether stretches past $\pm 2\sigma$ (shaded), the arbitrageur bets it snaps back. That bet is not a prediction about Charizard; it is a prediction about the relationship.

Notice what the arbitrageur is and is not claiming. It has **no opinion whatsoever** about whether Charizard is going up or down — it is short one and long the other, so a market-wide rally leaves it roughly flat. Its only bet is that *the relationship holds*. That is what “statistical arbitrage” means, and it is why such strategies can have small, steady returns that look nothing like those of the underlying instruments.

THE HONEST CAVEAT, WHICH IS ALSO THE RISK

Cointegration is an *assumption being tested*, not a guarantee. The correct procedure is to run a statistical test — Engle–Granger, which is an augmented Dickey–Fuller test on the regression residuals — and confirm the spread really is stationary before trading it. When a relationship that “always held” stops holding, the position that was hedged becomes two large naked bets at once, and the losses are famous. Our arb bot should therefore have a stop, and watching it get destroyed by a meta patch that breaks a relationship is a genuinely instructive thing to build.

5.11 Sharpe ratio: scoring the agents fairly

Finally, the leaderboard needs a metric, and total P&L is a bad one — it rewards whoever took the most risk, which over a short sample mostly rewards luck. The standard correction is return per unit of risk:

$$S = \frac{\bar{r} - r_f}{\sigma_r} \times \sqrt{N}$$

$\bar{r}$ mean return per period · r_f the risk-free rate (zero here) · σ_r its standard deviation · N periods per year, to annualise

An agent earning 5% with almost no variance is a better *business* than one earning 20% by nearly dying twice, and Sharpe is what says so. The $\sqrt{N}$ is there because mean return scales with time while standard deviation scales with $\sqrt{\text{time}}$ — so the ratio itself grows like $\sqrt{N}$, and you must divide out the sampling frequency before comparing anything.

WHERE SHARPE LIES TO YOU

Sharpe divides by standard deviation, which assumes risk is symmetric and adequately described by the second moment. We proved in 5.6 that our returns are fat-tailed — kurtosis 5.81 — so the fourth moment is exactly where the danger lives, and Sharpe cannot see it. A strategy that earns a little every day and loses everything once a year posts a magnificent Sharpe right up until it does not. We should report maximum drawdown alongside it, and being able to say *why* is worth more than the ratio itself.

Making the Pokémon layer do real work

If the theme is only a coat of paint, it is a liability. If it drives the price process, it is the best thing about the project. The difference is deliberate design.

The risk is obvious: an interviewer at a serious firm sees Pokémon on a CV and wonders whether they are looking at a toy. The defence is not to hide the theme — it is to make the theme *load-bearing*, so that every part of it is standing in for a real market phenomenon that would be harder to demonstrate otherwise.

THEMED FEATURE	REAL MECHANISM	WHAT IT LETS US MEASURE
Base stats → fundamental value	An asset with a knowable fair value	Whether prices actually converge to fundamentals, and how fast. You cannot do this with real equities because nobody knows the true value.
Meta patches "Water types buffed 15%"	A scheduled, quantified information shock	Adverse selection, in currency. Give agents different reaction latencies and measure exactly who profits from whom. This is the headline result of the whole project.
Evolution lines	A cointegrated basket	Statistical arbitrage that is genuinely justified rather than curve-fitted — we <i>know</i> the relationship exists because we built it.
Legendary rarity	A tiny free float	Slippage and spread as a function of depth, with everything else held constant. The cleanest possible microstructure experiment.
Type effectiveness	A correlation structure across instruments	Whether a portfolio-level hedge behaves as the correlation matrix says it should.

Fundamental value

Each species gets a fair value that agents can reason about — some knowing it precisely, some noisily, some not at all. A workable form:

$$V_0 = k \cdot \left(\frac{BST}{500} \right)^\alpha \cdot R \cdot T_t$$

BST base stat total — the in-game power number · **α** a convexity exponent around 1.5, so strong species are worth disproportionately more · **R** rarity multiplier (common 1, rare ~4, legendary ~40) · **T_t** the current meta's demand factor for that type — *the only time-varying term, and the one a patch moves*

To be clear about what this is: these are **design choices, not discovered laws**. We are writing the generating process, so we get to pick it. The interesting question is never whether the formula is "right" — it is whether a market populated by agents with partial, delayed and noisy knowledge of V_0 produces prices that behave like real ones. That is a real question with a real answer, and it is answerable precisely *because* we control the truth.

HOW TO PRESENT THIS ON PAPER

Put **"Limit Order Book & Matching Engine"** on the CV, with the theme as a parenthetical at most. Lead with the numbers: throughput, p99 latency, the data-structure change that bought the most. Then let the theme come out in conversation, where it works entirely in your favour — it is disarming, it is memorable, and it gives you a natural bridge into twenty minutes of genuine microstructure discussion. Theme on paper is a risk; theme in conversation is an asset.

What we build, in what order, and why

Six sub-projects, each independently shippable. We are starting the first slice of the first one, and nothing else.

The original spec is four phases and seven stages — realistically six to ten weeks. Designing all of it now would mean designing the UI against assumptions the engine has not yet tested. So it is decomposed, and each piece gets its own design, plan and build cycle:

	SUB-PROJECT	SHIPS	NEEDS
A	Engine core NOW	The book, the order types, cancel/modify, a Python reference, fuzz and determinism tests	—
B	Flow generator	Noise and momentum agents; a live tape in the terminal	A
C	Optimisation & benchmarks	v1–v3, the latency harness, the numbers	A, B
D	UI & gateway	WebSocket feed, ladder, tape, candles, latency panel	A, B
E	Smart agents	Avellaneda–Stoikov maker, arbitrageur, Pokémon fundamentals	B
F	Analysis	Stylized facts, impact curve, P&L attribution, the write-up	E

Note the critical path to the checkpoint your mentor asked for — throughput and latency numbers — is **A → B → C**. D, E and F are all valuable and none of them are on it.

Sub-project A, split in two

Even A is too much for one green commit, so it ships in two slices. The point of the split is that **A1’s tests are what make A2 safe** — the invariant checks and the Python differential harness are written before the fiddly order types they are meant to police.

SLICE CONTENTS

- A1** One instrument · limit and market orders · cancel · partial fills · price-time priority · the replay CLI · the Python reference · determinism and fuzz tests. **Green, pushed, demoable.**
- A2** IOC · FOK · post-only · modify with the priority rules from Part 3 · self-trade prevention · multiple instruments.

The shape of the code

libpokex/ types.hpp events.hpp book_concept.hpp book_v0_map.hpp matching.hpp	static lib · no I/O, no threads, no clock Price, Qty, OrderId, Side, Order, Sequence commands + events, and their text encoding the five operations, documented as a contract std::map + deque – the baseline, kept forever template<Book> – Part 2's loop, written ONCE
pokex-replay reference/engine.py tests/	CLI: commands.txt → events.txt. Pure. Deterministic. an independent reimplement, same file formats golden · property/fuzz · differential

Two decisions in there are worth restating, because they are the ones that pay off later:

- **The engine is a pure function.** Commands in, events out. No sockets, no threads, no database, no clock. Every later sub-project — the agents, the WebSocket gateway, even the lock-free ring buffer in v4 — attaches *around* that function without modifying it.
- **The book is swapped at compile time,** via `template<typename Book>` rather than a virtual base class. A virtual call is an indirect jump that defeats inlining and pollutes the branch predictor — so using one here would mean measuring our own abstraction instead of the data structure. The same test suite is instantiated against every version, which is what keeps four implementations from silently drifting apart and quietly invalidating the benchmark.

How we know it is correct

Three layers, weakest to strongest:

1. **Golden tests.** Hand-written scenarios with expected event streams. These catch semantic mistakes — the maker-price rule, residual handling — and they double as documentation of intended behaviour.
2. **Property tests / fuzzing.** Throw a million random *valid* command sequences at the engine and assert the invariants after every single one: the book never crosses; no order fills more than its size; and for every order, `filled + resting + cancelled == original`. Fuzzing does not need to know the right answer, only what must never happen — which is why it finds the bugs you did not think to write a test for.

3. **Differential testing.** Run the C++ engine and the Python reference on the same random flows and require byte-identical output. Two independent implementations agreeing is the strongest correctness evidence available to us, and it is only possible because of the determinism rule from Part 2.

What “done” means for A1

- Every test green, including a fuzz run of at least a million messages.
- The same input file replayed twice produces identical bytes.
- C++ and Python agree on ten thousand randomly generated flows.
- Clean under AddressSanitizer and UndefinedBehaviorSanitizer.
- A README that a stranger can follow to build and run it.

Note what is *not* on that list: any performance number at all. A1 is allowed to be slow. Making it fast is sub-project C, and doing it in that order is what makes the eventual speed claim believable rather than decorative.

THE BULLET WE ARE WORKING TOWARDS

Everything above is in service of being able to write one sentence and defend it for twenty minutes:

“Built a limit order book and matching engine in C++; replaced a `std::map` book with an array-indexed price ladder, an intrusive free-list allocator and an open-addressing ID index, taking sustained throughput from X to Y orders/sec and p99 matching latency from A to B μ s on an identical replayed order flow, verified byte-identical against a Python reference implementation.”

Note that every number in it is one **you measured**, and every claim is one the tests can prove. That is the difference between a project and a portfolio piece.

Every term, one line each

In alphabetical order, for when you are half-way through a conversation and need one fast.

Adverse selection

The tendency for your resting orders to be filled precisely by people who know something you do not. The market maker's central risk.

Aggressive / passive

An aggressive order trades immediately against the book (a taker). A passive order rests and waits (a maker).

Ask (offer)

A price at which someone is willing to sell. Asks sit above the spread.

Bid

A price at which someone is willing to buy. Bids sit below the spread.

Cache line

The 64-byte block the CPU always fetches as a unit. Data layout is about getting what you need next into the line you already have.

Cointegration

Two individually random-walking prices whose difference is stationary. The basis of statistical arbitrage.

Crossed book

An illegal state where the best bid is at or above the best ask. Can exist only for the instant before the engine resolves it.

Depth

How much quantity rests at each price level. Determines the cost of trading size.

Determinism

Same input bytes produce byte-identical output bytes, always. Requires no clocks, no randomness, no allocation-dependent iteration inside the engine.

False sharing

Two threads writing different variables that happen to share a cache line, causing them to fight over it. A slowdown with no logical cause.

FIFO

First in, first out. The queue discipline within a single price level.

Fill-or-kill (FOK)

Fill the entire quantity immediately or do nothing at all.

Immediate-or-cancel (IOC)

Take whatever is available right now; cancel the remainder rather than resting it.

Intrusive list

A linked list whose next/prev pointers live inside the element itself, so element and links share a cache line.

Inventory risk

The exposure a market maker carries from holding a position it did not want, acquired as a side effect of providing liquidity.

Kurtosis

The normalised fourth moment of a distribution. 3 for a Normal; higher means fatter tails. Financial returns are typically 4–20.

Limit order

An order with a price limit. Trades at that price or better, and rests if it cannot.

Liquidity

The ability to trade meaningful size, quickly, without moving the price. Has three dimensions: tightness, depth, resiliency.

Maker / taker

The maker's order was resting and provided liquidity; the taker arrived and consumed it. Trades print at the maker's price.

Market order

An order with no price limit. Always fills, at whatever the book costs.

Microprice

A fair-value estimate weighting each side's price by the *opposite* side's quantity, so book imbalance pulls the estimate.

Mid

The average of best bid and best ask. A convention, not a tradeable price.

Object pool / free list

Pre-allocating all the memory once and handing it out from a list, so the hot path never calls malloc.

Open addressing

A hash table that resolves collisions by probing nearby slots rather than chaining a linked list, avoiding pointer chasing.

Poisson process

Events arriving independently at a constant average rate; the gaps between them are exponentially distributed and memoryless.

Post-only

An order that must rest. Rejected if it would trade immediately, guaranteeing maker status and the associated rebate.

Price improvement

Getting a better price than your limit, because you traded at the resting order's price rather than your own.

Price-time priority

The matching rule: better price first, then earlier arrival. Time priority is the payment for patience.

Rebate

A small payment exchanges make to makers, funded by a fee on takers, to encourage resting liquidity.

Reservation price

In Avellaneda–Stoikov, the price at which a maker is genuinely indifferent — the mid, shifted against its own inventory.

Resiliency

How quickly the book refills after being consumed. The third and least-discussed dimension of liquidity.

Self-trade prevention

Refusing to match a participant against itself, because such trades fake the public volume signal (wash trading).

Sharpe ratio

Excess return divided by its standard deviation, annualised. Blind to fat tails, so quote maximum drawdown alongside it.

Slippage

The gap between the price you expected and the volume-weighted price you actually got. The cost of urgency and of size.

Spread

Best ask minus best bid. Not a fee — the price of immediacy.

SPSC ring buffer

Single-producer, single-consumer queue. Lets an ingress thread hand work to a matching thread without locks.

Stylized facts

Statistical properties found in essentially all real return series: fat tails, volatility clustering, near-zero return autocorrelation.

Tick

The smallest permitted price increment. Prices are integer counts of ticks, which is what makes an array-indexed book possible.

Top of book

The best bid and best ask together. The only two prices most participants ever look at.

Volatility

The standard deviation of returns. Not constant — it clusters.

Volatility clustering

Big moves follow big moves and quiet follows quiet. Visible as positive autocorrelation in $|\text{returns}|$.

VWAP

Volume-weighted average price. What a multi-level fill actually cost, in one number.

Wash trading

Trading with yourself to manufacture apparent volume. Illegal manipulation; the reason self-trade prevention exists.